

Phalcom Native Primitive Metadata and universe Package

Detailed Implementation Specification — Minimal Functional Vertical Slice

Status: Proposed implementation specification **Date:** 2026-08-15 **Target repository:** aureat/phalcom-lang

Scope: Full native primitive attribute metadata system, descriptor-driven primitive registration, VM-free native semantic surface generation, and a minimal built-in Package/universe namespace that is compatible with the future modules/packages/projects design. **Explicitly deferred:** Phaldoc syntax and parsing, full projects/modules/packages implementation, runtime realization of the future Phalcom type system, general user-defined native extensions.

1. Executive Summary

This change establishes one authoritative declaration site for every Rust-implemented Phalcom primitive and one canonical runtime namespace for every named built-in Phalcom class.

A native primitive will be declared directly on its Rust function:

```
[rust]
#[phalcom::primitive(
    Object,
    "methodFor(_)",

    params = [Symbol],
    returns = Option<Method>,
    types = "(Symbol) -> Option<Method>",

    raises = [],
    effects = pure,

    side = instance,
    visibility = public,
    stability = stable,

    abi = value,
    flow = value,
    trust = ordinary,
)]
pub fn object_method_for(
    vm: &mut VM,
    receiver: &Value,
    args: &[Value],
) -> PhResult<Value> {
    // ...
}
```

The declaration produces a compile-time PrimitiveDescriptor. The descriptor becomes the single source of truth for:

- primitive installation into the bootstrapped class tower;
- exact selector identity and dispatch side;
- visibility and runtime ABI;
- parameter tuple shape and parameter types;
- return type, including applied types such as Option<Method>;
- complete callable type contract;
- declared language-level raises and coarse semantic effects;
- return/control-flow provenance;
- stability and lifecycle metadata;
- intrinsic identity, when a compiler optimization explicitly recognizes the method;
- privileged-native audit classification;
- Rust source provenance;
- generated VM-free native interface metadata consumed by the LSP and future checker/doc tooling.

The design deliberately does **not** make types participate in ordinary primitive dispatch. Runtime primitive identity remains:

```
[text]
(owner, side, canonical selector)
```

Type information is semantic metadata, not selector encoding.

The same implementation introduces a real kernel `Package` class as a specialized `Module`, backed initially by the existing `ModuleObject` representation. A canonical built-in universe package object is allocated during bootstrap and populated with every named built-in class:

```
[phalcom]
universe.String
universe.Object
universe.Method
universe.BoundMethodFamily
universe.Package
```

`universe.String` is the exact same class object as the existing `String` binding. `BoundMethodFamily` may be absent from the user prelude while remaining available as `universe.BoundMethodFamily`.

This vertical slice intentionally avoids implementing filesystem package resolution, imports, projects, `package.ph`, dependency graphs, or package initialization. The object model and identity rules are chosen so those features can adopt the existing `Package` object later instead of replacing it.

2. Normative Language

The terms **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are used normatively for this implementation.

This document specifies implementation architecture rather than final user-facing Phalcom language syntax. Where future typing/module syntax is referenced, the implementation **MUST** preserve an extension seam rather than treating the current subset as a final grammar.

3. Current Repository Baseline

The implementation should be anchored to the current repository structure rather than layered beside it.

The relevant current components are:

Current area	Repository anchor	Current role	Change required
Primitive Rust functions	phalcom-core/src/primitive/*.rs	Native implementations	Add <code>#[phalcom::primitive(...)]</code> annotations
Primitive registration macros	phalcom-core/src/primitive/mod.rs	<code>primitive!</code> , <code>primitive_static!</code> , <code>primitive_shape!</code> , <code>rest/internal</code> variants	Replace with descriptor-driven installer after migration
Primitive installation table	phalcom-core/src/universe/primitives.rs	Manually binds Rust functions to classes/selectors	Collapse to registry installation
VM-free native surface	phalcom-native-surface/src/lib.rs	Handwritten <code>NATIVE_MEMBERS</code> /return metadata for LSP	Generate from descriptors
Core class creation	phalcom-core/src/universe/core_classes.rs	Builds <code>CoreClasses</code> and metaclass tower	Add <code>Package</code> ; add universe-key resolution
VM bootstrap	phalcom-core/src/vm/bootstrap.rs	Creates core module, installs primitives, runs <code>core.ph</code>	Create/populate/freeze universe; install descriptors
Module heap representation	phalcom-core/src/heap/module.rs	<code>ModuleObject</code> globals namespace	Make surface class configurable; add built-in/frozen namespace flags
Module behavior	phalcom-core/src/primitive/module.rs	Module creation guard and namespace member lookup through <code>dNU</code>	Reuse for <code>Package</code> inheritance
Module creation API	phalcom-core/src/vm/api.rs	<code>VM::create_module</code>	Pass <code>Module</code> surface class; add built-in package creator
Value class mapping	phalcom-core/src/value/mod.rs	Hardcodes <code>Object::Module -> Module</code>	Read the module object's stored surface class
Global/core fallback	dispatch/global cache implementation	User modules currently fall back to core globals	Gate fallback through explicit prelude-name metadata

The present system contains three parallel declarations for many native methods: the function/documentation in `primitive/*.rs`, the registration statement in `universe/primitives.rs`, and a corresponding entry in `phalcom-`

native-surface. The existing runtime then validates the runtime registration against the VM-free surface. The new design removes the need for this parallel maintenance by making the annotated Rust function authoritative.

4. Goals

The implementation **MUST** achieve all of the following in the initial landing:

1. A complete `#[phalcom::primitive(...)]` attribute surface capable of carrying all machine-readable metadata agreed before Phaldoc.
2. Structural parsing and validation of params, returns, and types.
3. Applied return types such as `option<Method>` represented without allocating runtime type objects.
4. Compile-time checking of the ordinary vs shape primitive Rust ABI.
5. Descriptor collection without a handwritten central primitive list.
6. Deterministic validation and installation of descriptors during VM bootstrap.
7. A stable VM-free semantic descriptor representation suitable for generated LSP metadata.
8. A stable `UniverseKey` identity for every canonical built-in class referenced by primitive metadata.
9. A real built-in `Package` class with `Package < Module`.
10. A singleton built-in `universe` package containing all named built-in classes.
11. `universe.X` must bind to the exact canonical class object, never a copy or wrapper.
12. Prelude membership must be represented independently from universe membership.
13. `BoundMethodFamily` must be expressible as universe-only metadata even if current bootstrap compatibility requires a staged prelude cleanup.
14. The design must be compatible with future modules/packages/projects without implementing those features now.
15. Phaldoc must remain a clean follow-up layer rather than being silently approximated by Rustdoc.

5. Non-Goals

This implementation **MUST NOT** attempt to implement:

- `project.toml`, projects, dependency universes, source roots, or project entypoints;
- `package.ph`, filesystem package discovery, package/module import syntax, re-exports, or relative imports;
- a general runtime package loader;
- full Phalcom typing/checking semantics;
- runtime construction or canonicalization of `AppliedType` objects such as `option<Method>`;
- automatic runtime validation of every typed primitive call in production;
- user-defined Rust extension primitives outside the core runtime;
- a final effect system;
- Phaldoc syntax, tags, examples, doctests, or documentation rendering;
- arbitrary bootstrap-phase ordering metadata;
- type-based primitive overload resolution.

6. Architectural Decisions

6.1 Primitive identity

A primitive's semantic identity **MUST** be:

```
[text]
PrimitiveKey = (UniverseKey owner, DispatchSide side, canonical selector)
```

The Rust function name **MUST NOT** be part of language identity.

Renaming:

```
[rust]
object_eq -> object_equality_primitive
```

must have no Phalcom-visible effect if the attribute is unchanged.

6.2 Types are metadata, not dispatch

The following declaration:

```
[rust]
params = [Number],
returns = Number,
types = "(Number) -> Number",
```

MUST NOT alter selector lookup. A primitive remains selected by owner/side/selector. Type metadata is consumed by tooling, future checking, debugging, documentation, and optimization validation only.

6.3 Type metadata is symbolic during bootstrap

`Option<Method>` MUST initially be represented as a static symbolic expression. Primitive installation MUST NOT require `AppliedType`, `TypeEnvironment`, or other future runtime typing objects to exist.

6.4 Universe membership is not prelude membership

`universe.String` and the prelude `String` refer to one class object. A class may be exported by universe without being an unqualified prelude binding.

6.5 `Package` is a specialized `Module`

The initial object model MUST be:

```
[text]
Package < Module < Object
```

The implementation SHOULD reuse `Object::Module(ModuleObject)` rather than introducing a second almost-identical namespace heap payload.

6.6 The built-in package is immutable after bootstrap

The universe namespace MUST be frozen after its canonical bindings are installed.

7. Target Attribute Surface

The canonical attribute grammar for this implementation is:

```
[rust]
#[phalcom::primitive(
  OWNER,
  "SELECTOR",

  params = [PARAMETER_TYPES...],
  returns = TYPE,
  types = "CALLABLE_TYPE",

  raises = RAISES_SPEC,
  effects = EFFECT_SPEC,

  side = instance | class,
  visibility = public | internal,
  stability = unspecified | experimental | stable,

  since = "SEMVER",
  deprecated_since = "SEMVER",
  replacement = "SELECTOR",

  abi = value | shape,
  flow = value | receiver | argument(INDEX) | never | unknown,

  intrinsic = INTRINSIC_ID,
  trust = ordinary | privileged,
)]
```

Only the first two arguments are positional. All remaining fields are named.

7.1 Defaults

Field	Default	Notes
side	instance	Class-side must be explicit
visibility	public	Internal namespace rule can force explicitness
stability	unspecified	Avoid claiming stability during early development
abi	value	Ordinary primitive Rust ABI
flow	value	Ordinary returned value, no stronger

		provenance claim
trust	ordinary	Privileged is audit metadata only
raises	omitted = unknown	raises = [] means known not to raise a language-level error
effects	omitted = unknown	effects = pure is a positive claim
since	absent	Optional lifecycle metadata
deprecated_since	absent	Optional lifecycle metadata
replacement	absent	Valid only with deprecation metadata
intrinsic	absent	Never inferred from selector

For the migration window, params, returns, and types MAY be temporarily optional behind an explicit compatibility mode. The final state of all public primitives SHOULD provide all three.

8. Parameter Metadata

8.1 Tuple-shaped argument model

params models the Phalcom argument tuple: an ordered positional lane followed by a labeled lane.

```
[rust]
params = [Object, Object, foo: SomeType]
```

normalizes to:

```
[text]
ParameterTupleSpec {
  positional: [Object, Object],
  labeled: [
    { label: "foo", ty: SomeType }
  ]
}
```

The receiver is **not** included in params.

8.2 Selector consistency

The selector parser and parameter parser MUST be cross-validated.

Example:

```
[rust]
#[phalcom::primitive(
  widget,
  "replace(_,with)",
  params = [Widget, with: Widget],
  returns = Widget,
  types = "(Widget, with: Widget) -> Widget",
)]
```

is valid.

This is invalid:

```
[rust]
params = [Widget, using: Widget]
```

because using does not match selector label with.

The macro MUST reject:

- positional-count mismatches;
- labeled-count mismatches;
- label spelling/order mismatches;
- a rest selector whose parameter metadata cannot represent its rest lane;
- duplicate labels.

8.3 Rest parameters

The normalized model MUST include a first-class rest description even if the initial surface syntax continues to derive rest details from the canonical selector.

```
[rust]
pub struct ParameterTupleSpec {
  pub positional: &'static [TypeExprSpec],
  pub labeled: &'static [LabeledParameterSpec],
```

```
pub rest: Option<RestParameterSpec>,
}
```

The attribute parser MAY initially permit rest metadata to be inferred from selectors such as `perform(_, ***)`; the descriptor MUST NOT force downstream consumers to reparse the selector to discover rest behavior.

Parameter source names distinct from selector labels are deferred. The internal representation MUST leave room to add them without changing the meaning of the existing labeled lane.

9. Type Expression Metadata

9.1 Shared symbolic vocabulary

The implementation MUST introduce a VM-free symbolic type-expression representation. It MUST be usable by the proc macro, generated native surface, LSP, and future checker without importing the VM.

Recommended minimal shape:

```
[rust]
#[derive(Clone, Copy, Debug, Eq, PartialEq, Hash)]
pub enum TypeExprSpec {
    Unknown,
    Nothing,
    SelfType,
    Universe(UniverseKey),
    Parameter(&'static str),
    Applied {
        origin: &'static TypeExprSpec,
        arguments: &'static [TypeExprSpec],
    },
    Union(&'static [TypeExprSpec]),
    Tuple(&'static ParameterTupleSpec),
}
```

This is a symbolic metadata AST, not the final runtime Type object model.

9.2 Built-in names resolve through `UniverseKey`

Within native metadata:

```
[rust]
String
BoundMethodFamily
Option<Method>
```

resolve against the universe catalog, not against the prelude.

Therefore a type may be referenced in primitive metadata even if ordinary Phalcom source would need qualification or a future import.

9.3 Applied types

`Option<Method>` MUST normalize to the equivalent of:

```
[text]
Applied {
    origin: Universe(Option),
    arguments: [Universe(Method)]
}
```

No `option(Method)` special case is permitted. The same machinery must later support `Result<T, E>`, `List<String>`, and other applied types.

9.4 Initial type-syntax subset

The metadata parser MUST support at least:

```
[text]
Name
universe.Name
Self
Nothing
Unknown
Name<A>
Name<A, B>
A | B
(A, B, label: C)
(A, label: B) -> R
<T>(T) -> Option<T>
<T, U>(T, using: U) -> U
```

The first implementation may intentionally omit intersections, aliases, bounds, variance, effects in callable types, const qualifiers, protocols not present in the universe catalog, and user-defined external type names.

The parser SHOULD live in a small VM-free syntax crate so the proc macro and future compiler/type tooling do not grow incompatible grammars.

Recommended crate:

```
[text]
phalcom-type-syntax
```

10. `params`, `returns`, and `types`

All three fields remain because they serve distinct consumers:

- `params`: selector-aligned argument structure;
- `returns`: direct first-class result type metadata;
- `types`: complete callable type contract including generic binders and relationships.

They MUST NOT be independent assertions.

For:

```
[rust]
params = [Symbol],
returns = Option<Method>,
types = "(Symbol) -> Option<Method>",
```

macro validation MUST prove structural equivalence between:

```
[text]
params == types.parameters
returns == types.return_type
```

For generic contracts:

```
[rust]
params = [T],
returns = Option<T>,
types = "<T>(T) -> Option<T>",
```

`T` is resolved against the callable binder.

An unbound uppercase identifier that is neither a binder nor a valid `UniverseKey` MUST fail compilation.

For this implementation, equivalence MAY be structural after normalization. More sophisticated type-theoretic equivalence can replace this comparison when the typing system becomes normative.

11. `raises`

`raises` describes language-level error contracts, not the Rust `PhResult` ABI.

The representation MUST distinguish:

```
[text]
omitted          => unknown
raises = []       => known no declared language-level raise
raises = [TypeError, RangeError]
```

Recommended representation:

```
[rust]
pub enum RaisesSpec {
    Unknown,
    Known(&'static [TypeExprSpec]),
}
```

Every declared raise type MUST resolve to a universe type and SHOULD ultimately be an `Error` subtype. During the initial implementation, subtype validation MAY remain a runtime/bootstrap assertion because the full type hierarchy is not available to the proc macro.

Internal VM failures, invariant panics, and Rust implementation errors are not part of the public `raises` contract.

12. Coarse Semantic Effects

This metadata is intentionally **not** a final Phalcom effect system.

Supported values:

```
[rust]
effects = pure
```

or:

```
[rust]
effects = [mutation, io, scheduling]
```

Recommended initial effect identifiers:

```
[text]
mutation
io
scheduling
reflection
nondeterminism
blocking
```

Omission means unknown.

pure is a strong positive claim and MUST be mutually exclusive with an effect list. Allocation is intentionally not modeled as an observable semantic effect in this MVP; a pure implementation may allocate internally.

Recommended representation:

```
[rust]
pub enum EffectSpec {
    Unknown,
    Pure,
    Known(&'static [NativeEffect]),
}
```

No optimizer is permitted to assume purity until the metadata has explicit test coverage and the optimizer has a separate design approving that use.

13. Visibility, Stability, and Lifecycle

13.1 Visibility

Supported values:

```
[text]
public
internal
```

Rules:

- selectors beginning with `_$` MUST explicitly provide visibility;
- `visibility = internal` SHOULD require the selector to use the internal `_$` namespace in the initial implementation;
- a `_$` selector may be deliberately dispatch-public only when the declaration explicitly says `visibility = public`.

This replaces special-case visibility logic currently embedded in primitive installation macros.

13.2 Stability

Supported values:

```
[text]
unspecified
experimental
stable
```

Visibility and stability are independent. `internal` is not a stability level.

13.3 Lifecycle fields

Optional fields:

```
[rust]
since = "0.3.0"
deprecated_since = "0.6.0"
replacement = "newMethod(_)"
```

Rules:

- version strings MUST parse as semantic versions;
- `deprecated_since` requires `since <= deprecated_since` when both are present;
- `replacement` requires deprecation metadata;

- replacement selector syntax **MUST** parse canonically;
- replacement initially refers to the same owner and side unless a later structured replacement key is introduced.

These fields are machine metadata. Human deprecation explanations are deferred to Phaldoc.

14. ABI and Return Flow

14.1 Primitive ABI

Supported values:

```
[text]
abi = value
abi = shape
```

`value` corresponds to the ordinary native primitive signature:

```
[rust]
pub type PrimitiveValueFn =
    fn(&mut VM, &Value, &[Value]) -> PhResult<Value>;
```

`shape` corresponds to shape-aware gateways:

```
[rust]
pub type PrimitiveShapeFn =
    fn(&mut VM, Value, ArgumentView) -> PhResult<CallOutcome>;
```

The attribute expansion **MUST** emit a Rust type coercion so ABI mismatch is a compiler error.

Example expansion fragment:

```
[rust]
const _: PrimitiveValueFn = object_eq;
```

A native rest selector **MUST** use a compatible shape-aware descriptor unless the runtime later gains another explicitly modeled rest ABI.

14.2 Return flow

Supported values:

```
[text]
value
receiver
argument(n)
never
unknown
```

`flow` carries provenance/control-flow information separate from static return type.

Examples:

```
[rust]
returns = Self,
flow = receiver,
```

and:

```
[rust]
returns = Nothing,
flow = never,
```

Validation rules:

- `argument(n)` index **MUST** be in range;
- when structurally decidable, `argument(n)` return type **SHOULD** equal the referenced parameter type;
- `never` **MUST** require `returns = Nothing`;
- `receiver` **SHOULD** require `returns = Self` or a structurally compatible owner type.

The MVP **MAY** downgrade the last rule to a diagnostic/test assertion if `Self` equivalence is not yet fully implemented.

15. Intrinsic Identity

Compiler-recognized behavior **MUST** be explicit and **MUST NOT** be inferred from selector spelling.

Optional syntax:

```
[rust]
intrinsic = BoolAnd
```

The ID resolves against a stable enum such as:

```
[rust]
pub enum NativeIntrinsicId {
    BoolAnd,
    BoolOr,
    BoolNot,
    // ... only truly compiler-recognized primitives
}
```

This field identifies the canonical ordinary send target that an optimizer/inliner may guard and deopt to. It does not create a second dispatch mechanism.

A method without compiler recognition omits `intrinsic`.

16. Native Trust Classification

Supported values:

```
[text]
ordinary
privileged
```

This is audit metadata, not Rust unsafe and not Phalcom visibility.

`privileged` SHOULD be used for primitives that can perform capabilities ordinary Phalcom code cannot, for example:

- mutate class/method tables directly;
- access hidden VM state;
- construct trusted runtime representations;
- bypass ordinary member visibility;
- participate in bootstrap authority.

CI SHOULD be able to emit a privileged-native census from descriptors.

17. Phaldoc Extension Seam — Deferred

This implementation MUST NOT specify or fake Phaldoc.

Rust `///` comments remain implementation documentation and MUST continue to work as Rustdoc. They MUST NOT automatically become the Phalcom user-facing API documentation contract.

The descriptor SHOULD reserve an optional documentation field at the Rust type level only if doing so does not freeze syntax:

```
[rust]
pub docs: Option<NativeDocRef>
```

but the proc macro MUST NOT accept a `phaldoc` field or `#[phalcom::phaldoc]` helper until the Phaldoc specification exists.

The later Phaldoc design should be able to attach parsed documentation metadata to the existing `PrimitiveDescriptor` without changing primitive identity, type contracts, or installation.

18. Crate and Module Layout

The recommended workspace additions are:

```
[text]
phalcom-type-syntax/
  src/lib.rs
  src/type_expr.rs
  src/callable.rs

phalcom-native-meta/
  src/lib.rs
  src/universe.rs
  src/types.rs
  src/primitive.rs
  src/manifest.rs

phalcom-native-macros/
  src/lib.rs
```

```

src/primitive.rs
src/parse.rs
src/validate.rs

phalcom-core/
src/native/mod.rs
src/native/descriptor.rs
src/native/registry.rs
src/native/install.rs
src/universe/catalog.rs
... existing primitive and bootstrap files ...

```

Dependency direction:

```

[graph BT
    phalcom_type_syntax[phalcom-type-syntax] --> phalcom_native_meta[phalcom-native-meta]
    phalcom_native_meta --> macros[macros]
    phalcom_native_meta --> native_surface_generated[native-surface/generated]
    macros --> phalcom_core[phalcom-core]

```

The VM-free crates MUST NOT depend on phalcom-core.

The proc macro crate MAY depend on syn, quote, proc-macro2, phalcom-type-syntax, and phalcom-native-meta.

phalcom-core may use a distributed static registry implementation such as linkme. The specific crate can be substituted, but the semantics in this specification are fixed: immutable descriptors, no static initializer side effects, and no language-visible ordering dependency.

19. Universe Identity Model

19.1 `UniverseKey`

Introduce a stable VM-free enum containing every named canonical built-in class that native metadata may reference.

Representative shape:

```

[rust]
#[derive(Clone, Copy, Debug, Eq, PartialEq, Hash, Ord, PartialOrd)]
pub enum UniverseKey {
    Object,
    Behavior,
    Class,
    Metaclass,

    Number,
    Int,
    Float,
    String,
    Bool,
    True,
    False,
    Symbol,

    Function,
    Closure,
    BoundMethod,
    Method,
    MethodFamily,
    BoundMethodFamily,
    Family,

    Option,
    Some,
    None,
    Unit,

    Iterable,
    List,
    Map,
    Set,
    Tuple,
    Record,
    Range,

```

```

    Bytes,

    Module,
    Package,
    System,
    Message,

    Error,
    MessageNotUnderstood,
    CannotYieldAcrossNativeFrame,
    UseAfterCloseError,

    Fiber,
    Resource,
}

```

The actual list MUST be generated/audited against `CoreClasses` so every named built-in class has exactly one key. Metaclass rows such as `Object` class do not require direct universe bindings; they remain reachable through ordinary class reflection.

19.2 Runtime resolution

`CoreClasses` MUST provide an exhaustive resolver:

```

[rust]
impl CoreClasses {
    pub fn resolve(&self, key: UniverseKey) -> ClassId {
        match key {
            UniverseKey::Object => self.object_class,
            UniverseKey::String => self.string_class,
            UniverseKey::Package => self.package_class,
            // ... exhaustive ...
        }
    }
}

```

Adding a `UniverseKey` without a runtime mapping MUST fail compilation through exhaustive matching.

20. Universe Binding Catalog

Introduce a VM-free binding catalog:

```

[rust]
pub struct UniverseBindingSpec {
    pub key: UniverseKey,
    pub name: &'static str,
    pub kind: UniverseBindingKind,
    pub exported: bool,
    pub prelude: bool,
}

```

Initial kind:

```

[rust]
pub enum UniverseBindingKind {
    Class,
}

```

The enum is deliberately extensible for future protocols, singleton services, constants, packages, and intrinsics.

Example entries:

```

[rust]
UniverseBindingSpec {
    key: UniverseKey::String,
    name: "String",
    kind: UniverseBindingKind::Class,
    exported: true,
    prelude: true,
}

UniverseBindingSpec {
    key: UniverseKey::BoundMethodFamily,
    name: "BoundMethodFamily",
    kind: UniverseBindingKind::Class,
    exported: true,
    prelude: false,
}

```

The `None` class requires special treatment: `universe.None` SHOULD refer to the canonical `None` class object, while the unqualified source-level `None` binding may continue to denote the immediate absence value. Therefore the `None` class's `prelude` flag SHOULD be `false`, with the immediate `None` value handled by a separate `prelude-value` entry.

21. Minimal `Package` Runtime Type

21.1 Class hierarchy

Add:

```
[text]
Package < Module
```

in `CoreClasses::create_core_classes` after `Module` exists.

`Package` MUST receive its own metaclass through the existing parallel metaclass rule.

21.2 Reuse `ModuleObject`

Do not add `Object::Package` in this vertical slice.

Instead, generalize `ModuleObject` so an `Object::Module` heap payload carries its surface class:

```
[rust]
pub struct ModuleObject {
    pub class: ClassId,
    pub name_sym: Symbol,
    pub name: String,
    pub path: String,
    // existing module state...

    pub builtin: bool,
    pub namespace_frozen: bool,
}
```

Change construction to require a class:

```
[rust]
pub fn new(
    class: ClassId,
    name: String,
    name_sym: Symbol,
    path: String,
    source: Option<Arc<String>>,
    builtin: bool,
) -> Self
```

Ordinary module allocation passes `vm.universe.classes.module_class`.

Built-in package allocation passes `vm.universe.classes.package_class`.

21.3 Value class resolution

Change:

```
[rust]
Object::Module(_) => vm.universe.classes.module_class,
```

into:

```
[rust]
Object::Module(module) => module.class,
```

This is the key compatibility seam: the same namespace payload can now surface as `Module` or any future `Module` subclass without adding another heap variant.

21.4 Call context

`Object::Module` backed packages continue to use:

```
[rust]
CallContext::Module { module: id }
```

No separate `CallContext::Package` is required. The context expresses namespace execution behavior, not the precise surface class.

21.5 Namespace lookup behavior

`Package` inherits `Module#doesNotUnderstand(_)`. Therefore:

```
[phalcom]
universe.String
```

works through the existing zero-argument module member lookup path against the namespace binding table.

No new package-specific member access mechanism is needed.

21.6 Freeze semantics

ModuleObject MUST gain a namespace freeze bit.

For a frozen namespace:

- declaring a new binding MUST fail;
- redefining/writing an existing binding through general namespace mutation MUST fail;
- VM bootstrap MAY populate bindings before setting the freeze bit;
- ordinary module source behavior remains unchanged because ordinary modules are not frozen.

Introduce an implementation-level error such as `RuntimeError::FrozenNamespace` or an equivalent existing error form.

The universe package MUST be frozen immediately after population.

22. Creating the `universe` Package

Add a private/bootstrap API:

```
[rust]
impl VM {
    fn create_builtin_package(&mut self, logical_name: &str) -> ObjRef {
        let name_sym = self.interner.intern(logical_name);
        let package = ModuleObject::new(
            self.universe.classes.package_class,
            logical_name.to_string(),
            name_sym,
            format!("<builtin:{logical_name}>"),
            None,
            true,
        );
        let id = self.heap.alloc(Object::Module(Box::new(package)));
        self.modules.insert(name_sym, id);
        id
    }
}
```

During bootstrap:

```
[text]
1. Universe::new creates the class tower, including Package.
2. VM is constructed.
3. VM allocates the `universe` Package object.
4. Every exported `UniverseBindingSpec` is resolved to its canonical class handle.
5. Those exact class handles are defined into the package namespace.
6. The namespace is frozen.
7. The bootstrap/core environment receives a global `universe` binding to the package object.
8. Primitive descriptors are installed.
9. Core source is executed under the existing required ordering.
```

The package object is registered in `VM::modules`, which is already part of the GC root set. No separate hidden GC root is required for the MVP.

23. Prelude Projection Without Implementing the Future Module System

Universe membership and prelude visibility must be separate now, but the current runtime's core-global fallback must remain compatible with `core.ph`.

The least invasive implementation is to keep all bootstrap implementation bindings in the core module while gating **user-module fallback** through a symbol whitelist.

Add to VM:

```
[rust]
pub prelude_names: HashSet<Symbol>,
```

This holds symbols only and is not a GC root.

During bootstrap:

- add every `UniverseBindingSpec` with `prelude = true`;
- add the global universe name;
- add special prelude value names such as immediate `None` when required;
- retain any temporary compatibility names explicitly documented by the migration.

When a user module misses a global and considers the core-module fallback, the dispatch/global-cache path **MUST** first require:

```
[rust]
vm.prelude_names.contains(&name_sym)
```

Code executing inside the core module continues to see its own full bootstrap globals directly. Therefore core implementation code can keep private class bindings without making them part of the user prelude.

This design is intentionally transitional. The future module/package implementation may replace `prelude_names` + `core fallback` with a real prelude namespace/import model without changing `UniverseKey`, universe package identity, or primitive descriptors.

Acceptance invariant:

```
[phalcom]
String == universe.String
```

and, once the staged prelude cleanup is enabled:

```
[text]
BoundMethodFamily // unresolved unqualified in user code
universe.BoundMethodFamily // resolves to canonical class
```

24. Primitive Metadata Data Structures

24.1 VM-free surface descriptor

Recommended structure in `phalcom-native-meta`:

```
[rust]
#[derive(Clone, Copy, Debug)]
pub struct PrimitiveSurfaceSpec {
    pub key: PrimitiveKey,
    pub visibility: NativeVisibility,
    pub stability: NativeStability,

    pub params: &'static ParameterTupleSpec,
    pub returns: &'static TypeExprSpec,
    pub callable: &'static CallableTypeSpec,

    pub raises: RaisesSpec,
    pub effects: EffectSpec,
    pub flow: ReturnFlowSpec,

    pub since: Option<&'static str>,
    pub deprecated_since: Option<&'static str>,
    pub replacement: Option<&'static str>,

    pub intrinsic: Option<NativeIntrinsicId>,
    pub trust: NativeTrust,
}
```

24.2 Runtime descriptor

In `phalcom-core`:

```
[rust]
#[derive(Clone, Copy)]
pub struct PrimitiveDescriptor {
    pub surface: &'static PrimitiveSurfaceSpec,
    pub abi: PrimitiveAbi,
    pub entry: PrimitiveEntry,
    pub source: NativeSourceSpec,
}
```

Function pointer union:

```
[rust]
pub enum PrimitiveEntry {
    Value(PrimitiveValueFn),
    Shape(PrimitiveShapeFn),
}
```

Source provenance:

```
[rust]
pub struct NativeSourceSpec {
    pub module_path: &'static str,
    pub rust_name: &'static str,
    pub file: &'static str,
    pub line: u32,
}
```

24.3 Primitive key

```
[rust]
pub struct PrimitiveKey {
    pub owner: UniverseKey,
    pub side: NativeDispatch,
    pub selector: &'static str,
}
```

No ordinal ID is assigned from registry order.

25. Procedural Macro Parsing Pipeline

For every annotated function, the macro MUST execute the following pipeline in order.

Step 1 — Parse owner and selector

- owner token becomes a UniverseKey path;
- selector string is parsed by the shared canonical selector parser;
- canonical spelling is produced and compared with source spelling;
- noncanonical spelling is rejected with the canonical suggestion.

Step 2 — Parse argument tuple metadata

- parse positional types;
- parse labeled entries;
- parse/infer rest lane;
- resolve generic binder references when types declares them.

Step 3 — Parse return expression

Parse returns = ... using the same symbolic type AST.

Step 4 — Parse complete callable type

Parse the types string with phalcom-type-syntax and require a callable type.

Step 5 — Cross-check selector and `params`

Validate arity, labels, method/getter/setter shape, and rest behavior.

Step 6 — Cross-check type contract

Require structural normalized equivalence:

```
[text]
params == callable.params
returns == callable.returns
```

Step 7 — Parse semantic metadata

Validate raises, effects, flow, lifecycle, intrinsic, trust, side, visibility, and stability.

Step 8 — Validate Rust item shape

Reject:

- async fn;
- generic Rust functions;
- unsupported extern ABI;
- methods nested in impls if the registry machinery only supports free functions in the MVP;
- wrong Rust primitive function type.

Step 9 — Emit original function

The function remains callable/testable normally.

Step 10 — Emit static metadata

Emit static type/parameter/callable objects and a PrimitiveSurfaceSpec.

Step 11 — Emit ABI coercion

Example:

```
[rust]
const _: crate::native::PrimitiveValueFn = object_method_for;
```

Step 12 — Contribute runtime descriptor

Emit a distributed static registry entry containing the surface pointer, function pointer, ABI, and source provenance.

26. Descriptor Registry

Use an immutable distributed slice or equivalent linker-collected registry.

Recommended shape:

```
[rust]
#[linkme::distributed_slice]
pub static PRIMITIVES: [PrimitiveDescriptor];
```

Each macro contributes:

```
[rust]
#[linkme::distributed_slice(crate::native::registry::PRIMITIVES)]
static __PHALCOM_PRIMITIVE_OBJECT_METHOD_FOR: PrimitiveDescriptor = ...;
```

Requirements:

- registry enumeration order is explicitly unspecified;
- installer MUST sort descriptors by semantic key before validation/installation;
- source/link order MUST NOT affect Phalcom semantics;
- no constructor/static-initializer function may mutate the VM;
- descriptors are immutable data only.

Duplicate semantic keys cannot reliably be diagnosed by an isolated proc macro across the whole crate. They MUST be detected before installation and by a dedicated test/CI check.

27. Registry Validation

Before mutating any class method table, bootstrap MUST validate the complete registry.

Validation includes:

16. unique (owner, side, selector) keys;
17. every owner resolves through CoreClasses;
18. every universe type referenced by metadata exists in the universe catalog;
19. every declared applied-type origin has legal arity if that information is known;
20. every raises type is a valid known type and, where possible, an Error descendant;
21. every intrinsic ID appears at most once unless explicitly designed otherwise;
22. internal selector/visibility invariants;
23. deprecation replacement selector syntax;
24. no impossible flow contract;
25. no unsupported ABI/rest combination.

Validation MUST finish before the first method installation so a bad registry cannot leave the runtime partially mutated.

28. Descriptor-Driven Primitive Installation

The installer replaces the six existing primitive macro families with orthogonal descriptor fields.

Pseudo-code:

```
[rust]
pub fn install_all(vm: &mut VM) -> PhResult<()> {
    let mut descriptors = PRIMITIVES.iter().copied().collect::<Vec<_>>();
    descriptors.sort_by_key(|d| d.surface.key.sort_key());

    validate_registry(vm, &descriptors)?;
```

```

    for descriptor in descriptors {
        install_one(vm, descriptor)?;
    }

    Ok(())
}

```

install_one:

```

[rust]
fn install_one(vm: &mut VM, descriptor: PrimitiveDescriptor) -> PhResult<()> {
    let owner = vm.universe.classes.resolve(descriptor.surface.key.owner);

    let target = match descriptor.surface.key.side {
        NativeDispatch::Instance => owner,
        NativeDispatch::Class => vm.heap.class(owner).class,
    };

    let selector = vm.get_or_intern(descriptor.surface.key.selector);

    let mut method = match descriptor.entry {
        PrimitiveEntry::Value(f) => {
            MethodObject::new_primitive(
                selector,
                descriptor.runtime_signature(),
                f,
                owner,
            )
        }
        PrimitiveEntry::Shape(f) => {
            MethodObject::new_shape_primitive(
                selector,
                descriptor.runtime_method_signature(selector),
                f,
                owner,
            )
        }
    };

    method.visibility = descriptor.surface.visibility.into();
    method.access_owner = descriptor.internal_access_owner(owner);

    let method_id = vm.heap.alloc(Object::Method(Box::new(method)));
    vm.install_native_method_binding(target, selector, method_id)?;

    Ok(())
}

```

Rest-layout construction **MUST** be generated from the parsed selector/parameter descriptor rather than handwritten macro arguments.

The installer **SHOULD** reuse the existing validated method-binding path that already maintains the primary selector dictionary, rest-family index, and world_version consistently.

29. Migration of Existing Primitive Macros

Migration **MUST** be staged so failures are attributable.

Phase M1 — Infrastructure only

- add metadata/syntax/macro crates;
- add registry and installer behind tests;
- keep existing primitive macros authoritative;
- annotate a small representative set with a non-installing validation mode if necessary.

Representative methods:

[text]	
Object#name	getter, simple return
Object#==()	ordinary positional method
Object#methodFor()	Option<Method>
Number.class#new()	class-side method
Object#perform(, **)	rest + shape ABI
Object#_invariantEnter()	internal visibility
Bool#and()	intrinsic example

Phase M2 — Dual-path representative installation

Move the representative methods to descriptor-driven installation and remove their handwritten macro calls.

Boot tests must prove exact selector/visibility/side parity.

Phase M3 — Convert one primitive module at a time

Suggested order:

```
[text]
object
number/int/float
string/symbol/bool
option/some
method/family/method_family
collections
module/system
fiber/resource
remaining internal primitives
```

Phase M4 — Delete central registration statements

Once every primitive is descriptor-installed, collapse `Universe::install_primitives` to the registry installer.

Phase M5 — Delete obsolete primitive macros

Remove:

```
[text]
primitive!
primitive_shape!
primitive_rest!
primitive_internal!
primitive_static!
primitive_static_internal!
```

only after no call sites remain.

Phase M6 — Remove runtime-vs-native-surface drift validation

The current comparison exists because there are two declarations. Once `phalcom-native-surface` is generated from descriptors, replace that validator with descriptor self-validation and generated-file freshness checks.

30. Generated VM-Free Native Surface

`phalcom-native-surface` SHOULD remain VM-free because the LSP should not need to link the runtime.

Its handwritten native-member table SHOULD become generated output.

Recommended generator flow:

```
[text]
annotated Rust functions
-> linked PrimitiveDescriptor registry
-> explicit generator binary/tool
-> generated VM-free Rust metadata
-> phalcom-native-surface
-> LSP/checker/doc tooling
```

Do not make the procedural macro write workspace files.

Recommended command:

```
[text]
cargo xtask native-surface generate
cargo xtask native-surface generate --check
```

If the repository does not yet have an `xtask`, a small dedicated generator binary is acceptable initially.

Generated records MUST exclude VM function pointers and MUST include:

- schema version;
- primitive key;
- visibility;
- stability/lifecycle;
- params;
- return type;
- complete callable type;
- raises;
- effects;
- flow;
- intrinsic ID;
- trust classification;

- Rust provenance;
- owner universe metadata needed for symbol construction.

Phaldoc content is absent until specified.

31. Manifest Versioning

Introduce a native metadata schema version immediately.

Example:

```
[rust]
pub const NATIVE_SURFACE_SCHEMA_VERSION: u32 = 1;
```

If JSON is emitted for debugging/external tooling:

```
[json]
{
  "schema": 1,
  "universe": [],
  "primitives": []
}
```

A schema bump is required for incompatible generated-manifest shape changes.

This schema version is a tooling contract, not a language version.

32. Bootstrap Sequence

The target VM bootstrap ordering is:

```
[text]
A. Allocate and wire apex class/metaclass tower.
B. Create remaining core classes, including Module and Package.
C. Construct VM state.
D. Allocate built-in `universe` Package object.
E. Populate universe package with canonical class objects.
F. Freeze universe namespace.
G. Create/register bootstrap core module.
H. Define bootstrap class globals needed by core implementation.
I. Define `universe` in core.
J. Build prelude-name whitelist from metadata + special values.
K. Validate complete primitive descriptor registry.
L. Install primitive descriptors.
M. Finalize native base-name indexes.
N. Compile and execute core source.
O. Snapshot post-core optimization guard state.
P. Verify universe/class/primitive invariants.
```

The implementation **MUST** preserve any existing ordering dependency requiring primitives to be installed before `core.ph` executes.

Runtime type-expression realization is **not** inserted into this sequence yet. Symbolic metadata is sufficient.

33. File-by-File Implementation Map

Workspace `Cargo.toml`

Add:

```
[text]
phalcom-type-syntax
phalcom-native-meta
phalcom-native-macros
```

and registry dependency for `phalcom-core`.

`phalcom-native-meta/src/universe.rs`

Implement:

```
[text]
UniverseKey
UniverseBindingKind
UniverseBindingSpec
NATIVE_SURFACE_SCHEMA_VERSION
```

`phalcom-native-meta/src/types.rs`

Implement:

```
[text]
TypeExprSpec
LabeledParameterSpec
RestParameterSpec
ParameterTupleSpec
TypeParameterSpec
CallableTypeSpec
```

`phalcom-native-meta/src/primitive.rs`

Implement semantic enums/records:

```
[text]
PrimitiveKey
NativeDispatch
NativeVisibility
NativeStability
RaisesSpec
NativeEffect
EffectSpec
ReturnFlowSpec
NativeIntrinsicId
NativeTrust
PrimitiveSurfaceSpec
NativeSourceSpec VM-free subset as appropriate
```

`phalcom-type-syntax`

Implement parser/normalizer tests for the agreed subset.

If existing selector parsing cannot be reused by proc macros, factor canonical selector syntax into a VM-free common module instead of duplicating it.

`phalcom-native-macros`

Implement the primitive proc macro and compile-fail diagnostics.

Do not implement Phaldoc.

`phalcom-core/src/native/descriptor.rs`

Implement runtime ABI aliases, `PrimitiveEntry`, and `PrimitiveDescriptor`.

`phalcom-core/src/native/registry.rs`

Define distributed slice and full-registry validation.

`phalcom-core/src/native/install.rs`

Implement deterministic installation and runtime signature/rest construction.

`phalcom-core/src/universe/core\_classes.rs`

- add `package_class`;
- create `Package < Module`;
- include it in handle iteration/invariant plumbing;
- expose `CoreClasses::resolve(UniverseKey)`.

`phalcom-core/src/heap/module.rs`

- add `class: ClassId`;
- add `builtin: bool`;
- add `namespace_frozen: bool`;
- update constructor;
- enforce freeze in mutation paths.

`phalcom-core/src/value/mod.rs`

Resolve `Object::Module(module)` through `module.class`.

`phalcom-core/src/vm/api.rs`

- update ordinary module creation to pass `module_class`;
- add private built-in package creation;
- optionally add bootstrap-only namespace-definition helper.

`phalcom-core/src/vm/bootstrap.rs`

- allocate/populate/freeze universe;
- bind universe into core;
- build prelude-name whitelist;
- switch primitive installation to descriptors as migration progresses.

`phalcom-core/src/vm/mod.rs`

Add `prelude_names: HashSet<Symbol>` and update all exhaustive VM destructures (notably GC root classification) to classify it as symbol-only/non-root state.

Global dispatch/cache implementation

Gate core fallback through `prelude_names` for non-core modules. Preserve cache invalidation semantics.

`phalcom-core/src/primitive/\*.rs`

Annotate every native primitive.

`phalcom-core/src/universe/primitives.rs`

Delete manual imports and registrations incrementally; end state contains only registry installation/validation glue or disappears into `native/install.rs`.

`phalcom-native-surface`

Replace handwritten member list with checked-in generated data.

34. Diagnostics Specification

The proc macro and registry validator MUST emit actionable diagnostics.

Error	Required diagnostic content
Invalid selector	Owner/function, invalid selector, parse reason
Noncanonical selector	Original spelling and canonical replacement
Unknown owner	Invalid universe owner identifier
Params arity mismatch	Selector arity vs declared positional/labeled counts
Label mismatch	Selector label and params label at mismatch position
types parse failure	Original type string and parser location
Params/types mismatch	Normalized parameter tuple from both declarations
Return/types mismatch	Normalized return expression from both declarations
Unbound type variable	Variable name and available binders
Unknown universe type	Type name and owner function
Internal namespace ambiguity	<code>_\$</code> selector requires explicit visibility
ABI mismatch	Declared ABI and expected Rust function type
Rest/ABI mismatch	Rest selector requires supported rest ABI
Invalid flow argument	Index and parameter count
never mismatch	Require returns = Nothing
Invalid effect combination	pure cannot coexist with effect list
Invalid lifecycle	Bad semver/order/replacement rule
Duplicate primitive key	Both Rust source locations
Duplicate intrinsic mapping	Intrinsic ID and conflicting primitive keys
Frozen universe mutation	Namespace name and attempted binding

Diagnostics should identify the Phalcom semantic name first and Rust implementation location second.

Example:

```
[text]
error: primitive type contract is inconsistent
  primitive: Object#methodFor(_)
  rust:      primitive::object::object_method_for

  params:      [String]
  types params: [Symbol]

  expected structurally equivalent parameter tuples
```

35. Testing Strategy

35.1 Type-syntax parser tests

Cover:

```
[text]
Object
Option<Method>
Result<String, Error>
A | B
(Object, foo: String)
(Object, foo: String) -> Bool
<T>(T) -> Option<T>
universe.BoundMethodFamily
```

Include malformed input and source-offset diagnostics.

35.2 Proc-macro compile-pass tests

Use trybuild or equivalent for:

- ordinary value primitive;
- class-side primitive;
- internal primitive;
- generic callable contract;
- Option<Method> return;
- shape/rest primitive;
- intrinsic/trust/lifecycle metadata.

35.3 Proc-macro compile-fail tests

Cover every diagnostic class in Section 34.

35.4 Registry unit tests

- deterministic sort independent of link order;
- duplicate key rejection;
- intrinsic uniqueness;
- full universe owner resolution;
- rest-layout construction parity with current runtime;
- visibility/access-owner parity.

35.5 Runtime parity tests during migration

For every migrated primitive, compare:

```
[text]
owner class
metaclass/instance side
selector
signature kind
arity/rest layout
visibility
access owner
method kind
```

against expected pre-migration behavior.

35.6 Universe package tests

Required:

```
[phalcom]
assert(universe.class === Package)
assert(universe.String === String)
assert(universe.Object === Object)
assert(universe.Method.class === Metaclass) // or equivalent class-object assertion
```

Rust-level tests MUST verify `universe.BoundMethodFamily` resolves to `CoreClasses::bound_method_family_class`.

Verify `universe.None` is the `None` class object while the ordinary `None` expression/global remains the immediate absence value according to existing semantics.

35.7 Freeze tests

After bootstrap:

- attempting to define a new universe binding fails;
- attempting to replace `universe.String` fails;
- force GC does not invalidate universe package or contained class identities.

35.8 Prelude gating tests

When prelude separation is enabled:

```
[text]
String          -> resolves
universe.String -> resolves, identical
BoundMethodFamily -> does not resolve unqualified
universe.BoundMethodFamily -> resolves
```

Core bootstrap code must still be able to use its private core globals.

35.9 Generated-surface tests

- generator output is deterministic;
- `--check` reports no diff on a clean tree;
- every runtime descriptor has one generated surface record;
- no generated record exists without a runtime descriptor;
- schema version is present.

36. Acceptance Criteria

The change is considered minimally complete only when all of the following are true.

Attribute and metadata

- ☐ ``#[phalcom::primitive]`` accepts the full field surface in Section 7.
- ☐ ``params``, ``returns``, and ``types`` parse and cross-check.
- ☐ ``Option<Method>`` is represented as an applied symbolic type.
- ☐ labeled parameter tuples are supported.
- ☐ generic binder references are supported in the metadata subset.
- ☐ ``raises``, effects, lifecycle, ABI, flow, intrinsic, and trust metadata are represented.
- ☐ Rust ABI mismatch is a compile error.
- ☐ selector/params mismatch is a compile error.
- ☐ internal selector visibility rules are enforced.

Registry and installation

- ☐ Annotated primitives contribute immutable descriptors automatically.
- ☐ registry order has no semantic effect.
- ☐ duplicate primitive keys fail before method-table mutation.
- ☐ representative value/class/shape/internal/rest primitives install through descriptors.
- ☐ after migration, all primitives install through descriptors.
- ☐ old primitive registration macros are deleted or unused.

Universe and Package

- ☐ ``Package`` exists as a kernel class with ``Package < Module``.
- ☐ package instances can reuse ``Object::Module`` with a stored surface class.
- ☐ canonical built-in ``universe`` package exists.
- ☐ every named core class has a ``UniverseKey`` and exported universe binding policy.
- ☐ ``universe.X`` references exact canonical class handles.
- ☐ universe namespace freezes after bootstrap.
- ☐ ``universe`` is reachable as a global built-in name.
- ☐ prelude metadata is independent of universe membership.
- ☐ at least ``BoundMethodFamily`` is modeled as universe-only.

Tooling

- ☐ VM-free native surface is generated from descriptors.
- ☐ generated output carries schema version and full structural metadata.
- ☐ LSP can consume the generated surface without linking ``phalcom-core``.
- ☐ checked-in generated output has a CI freshness check.

Deferred boundary

- ☐ no Phaldoc syntax is introduced accidentally.
- ☐ no runtime ``AppliedType`` construction is required.
- ☐ no project/package filesystem resolution is implemented.

37. Recommended Delivery Phases

Phase 0 — Syntax and metadata foundation

Deliverables:

- `phalcom-type-syntax`;
- `phalcom-native-meta`;
- `UniverseKey`;
- primitive semantic data structures;
- parser/unit tests.

Exit condition: symbolic metadata can represent every target attribute example without the VM.

Phase 1 — Proc macro and descriptor registry

Deliverables:

- `phalcom-native-macros`;
- `#[phalcom::primitive]`;
- distributed registry;
- compile-pass/fail tests;
- representative annotations without broad migration.

Exit condition: representative descriptors compile and registry validation works.

Phase 2 — Descriptor-driven installation

Deliverables:

- runtime descriptor/installer;
- migrate representative primitives;
- parity tests.

Exit condition: VM boots with representative methods installed only through descriptors.

Phase 3 — ``Package`` and ``universe``

Deliverables:

- `Package < Module`;
- generalized `ModuleObject.class`;
- built-in universe package population/freeze;

- universe/global/prelude metadata;
- identity tests.

Exit condition: `universe.String === String` and universe-only class access works.

Phase 4 — Full primitive migration

Deliverables:

- annotate all primitive modules;
- remove handwritten registrations;
- delete obsolete macros;
- delete runtime-vs-handwritten-surface drift checker.

Exit condition: no primitive installation metadata exists outside annotations/descriptors.

Phase 5 — Generated native surface

Deliverables:

- generator;
- generated `phalcom-native-surface`;
- LSP integration parity;
- CI `--check`.

Exit condition: LSP receives at least the same native member coverage as before, now with richer type/effect/lifecycle metadata.

Phase 6 — Prelude enforcement cleanup

Deliverables:

- prelude-name whitelist in global fallback;
- audit core private vs user-prelude names;
- `BoundMethodFamily` universe-only behavior.

This can land with Phase 3 if the core source requires no incompatible hidden bindings; otherwise stage it immediately after the descriptor/universe landing.

38. Risks and Mitigations

Risk: metadata becomes a second type system

Mitigation: keep `TypeExprSpec` explicitly symbolic and derived from a shared syntax parser. Do not invent native-only constructs such as `option(Method)`.

Risk: three type fields drift

Mitigation: compile-time structural equivalence checks between params, returns, and types.

Risk: distributed registry order becomes accidental ABI

Mitigation: sort by semantic `PrimitiveKey`; never expose registry ordinals.

Risk: proc macro filesystem side effects

Mitigation: proc macro emits descriptors only. Generated files are produced by an explicit tool.

Risk: package implementation conflicts with future modules

Mitigation: `Package < Module`, reuse namespace payload, stable object identity, no filesystem semantics now.

Risk: universe package is garbage collected

Mitigation: register it in the VM's rooted module map and bind it from core.

Risk: ``None`` class conflicts with ``None`` value

Mitigation: `universe.None` is the class object; prelude `None` remains the immediate value; do not project the class under the same unqualified binding.

Risk: core module and prelude are conflated

Mitigation: gate user fallback with `prelude_names`, allowing core implementation globals to remain private.

Risk: effect metadata is overtrusted

Mitigation: default to unknown; require explicit pure; prohibit optimizer assumptions until separately approved.

Risk: error classes are incomplete

Mitigation: raises defaults to unknown. Annotate only contracts whose surface error types exist; expand universe error classes incrementally.

39. Future-Compatible Extension Points

The initial implementation should intentionally leave these additive seams:

26. `UniverseBindingKind` can gain `Protocol`, `Singleton`, `Constant`, and `Package` entries.
27. `TypeExprSpec` can be replaced/mapped to canonical runtime `Type` objects once typing lands.
28. `PrimitiveSurfaceSpec` can gain parsed Phaldoc without identity changes.
29. Parameter metadata can gain source parameter names independently of selector labels.
30. `Package` can gain logical parent/package identity, exports, initialization state, and project ownership.
31. The universe package can become a normal built-in dependency in the future project resolver.
32. `prelude_names` can be replaced by the future module/prelude import mechanism.
33. Native descriptors can later be reused by a Rust extension/FFI system without changing core primitive syntax.
34. Native contract checking can be enabled in checker/debug modes without imposing production-call overhead.
35. Generated manifests can grow documentation, source links, or richer effect/type relationships under schema versioning.

40. Reference Attribute Examples

40.1 Simple getter

```
[rust]
#[phalcom::primitive(
  Object,
  "name",
  params = [],
  returns = String,
  types = "() -> String",
  raises = [],
  effects = pure,
  stability = stable,
)]
pub fn object_name(...) -> PhResult<Value> {
  // ...
}
```

40.2 Equality

```
[rust]
#[phalcom::primitive(
  Object,
  "==(_)",
  params = [Object],
  returns = Bool,
  types = "(Object) -> Bool",
  raises = [],
  effects = pure,
  stability = stable,
)]
pub fn object_eq(...) -> PhResult<Value> {
  // ...
}
```

40.3 Optional result

```
[rust]
#[phalcom::primitive(
  Object,
  "methodFor(_)",
```

```

    params = [Symbol],
    returns = Option<Method>,
    types = "(Symbol) -> Option<Method>",
    raises = [],
    effects = pure,
    stability = stable,
  )]
pub fn object_method_for(...) -> PhResult<Value> {
  // ...
}

```

40.4 Labeled tuple-shaped parameters

```

[rust]
#[phalcom::primitive(
  Example,
  "combine(_,_,foo)",
  params = [Object, Object, foo: SomeType],
  returns = Option<ResultType>,
  types = "(Object, Object, foo: SomeType) -> Option<ResultType>",
  raises = [Error],
  effects = unknown,
  stability = experimental,
)]
pub fn example_combine(...) -> PhResult<Value> {
  // ...
}

```

effects = unknown may be represented by omission if the macro grammar does not expose the literal unknown; choose one spelling and test it consistently.

40.5 Class-side constructor

```

[rust]
#[phalcom::primitive(
  Number,
  "new(_)",
  params = [Object],
  returns = Number,
  types = "(Object) -> Number",
  side = class,
  raises = [Error],
  effects = pure,
  stability = stable,
)]
pub fn number_class_new(...) -> PhResult<Value> {
  // ...
}

```

40.6 Internal primitive

```

[rust]
#[phalcom::primitive(
  Object,
  "$invariantEnter()",
  params = [],
  returns = Unit,
  types = "() -> Unit",
  visibility = internal,
  effects = [mutation],
  trust = privileged,
)]
pub fn object_invariant_enter(...) -> PhResult<Value> {
  // ...
}

```

40.7 Shape/rest primitive

```

[rust]
#[phalcom::primitive(
  Object,
  "perform(_,***)",
  params = [Symbol],
  returns = Object,
  types = "(Symbol, ...) -> Object",
  abi = shape,
  raises = [Error],
  effects = [reflection],
  trust = privileged,
)]
pub fn object_perform_shape(
  vm: &mut VM,
  receiver: Value,
  args: ArgumentView,
) -> PhResult<CallOutcome> {

```

```
// ...
}
```

The exact textual callable representation of rest lanes (. . . above) MUST be finalized by phalcom-type-syntax tests before migration. The descriptor's structural rest lane is authoritative.

40.8 Compiler intrinsic target

```
[rust]
#[phalcom::primitive(
  Bool,
  "and(_)",
  params = [Object],
  returns = Object,
  types = "(Object) -> Object",
  intrinsic = BoolAnd,
  effects = unknown,
  stability = stable,
)]
pub fn bool_and(...) -> PhResult<Value> {
  // ...
}
```

41. Example Generated Native Surface Record

A generated VM-free record for `Object#methodFor(_)` should be conceptually equivalent to:

```
[rust]
NativeMember {
  key: PrimitiveKey {
    owner: UniverseKey::Object,
    side: NativeDispatch::Instance,
    selector: "methodFor(_)",
  },
  visibility: NativeVisibility::Public,
  stability: NativeStability::Stable,
  params: ParameterTupleSpec {
    positional: &[TypeExprSpec::Universe(UniverseKey::Symbol)],
    labeled: &[],
    rest: None,
  },
  returns: TypeExprSpec::Applied {
    origin: &TypeExprSpec::Universe(UniverseKey::Option),
    arguments: &[TypeExprSpec::Universe(UniverseKey::Method)],
  },
  callable_source: "(Symbol) -> Option<Method>",
  raises: RaisesSpec::Known(&[]),
  effects: EffectSpec::Pure,
  flow: ReturnFlowSpec::Value,
  intrinsic: None,
  trust: NativeTrust::Ordinary,
  source: NativeSourceSpec {
    module_path: "phalcom_core::primitive::object",
    rust_name: "object_method_for",
    file: "phalcom-core/src/primitive/object.rs",
    line: 0,
  },
}
```

Line number is populated by macro expansion in real output.

42. Definition of Done

The implementation is done when a runtime developer can open any native primitive function and determine its complete machine-readable Phalcom interface without searching a second registration file; the VM can bootstrap every primitive from those declarations; the LSP can consume generated metadata without linking the VM; and every canonical built-in class has a stable identity under the built-in universe package.

The key architectural invariants at completion are:

```
[text]
One primitive declaration -> all native semantic/install/tooling metadata.
One canonical built-in class object -> one UniverseKey -> one universe binding.
Prelude exposure -> a projection, never the source of built-in identity.
Package identity -> established now, filesystem/package loading -> deferred.
Phaldoc -> additive future layer, not entangled with primitive registration.
```